



CSE 4202: Structured Programming II Lab

Lecture 4 — File I/O

Syed Rifat Raiyan

Lecturer

Department of Computer Science & Engineering
Islamic University of Technology, Dhaka, Bangladesh

Email: rifatraiyan@iut-dhaka.edu

Lecture Plan

The agenda for today

- Understand what files are, their types, and types of operation
- Understand why we need file I/O
- Abstraction of files in C
- Some common file I/O functions in C
- Implement the **cat** and **cp** Linux terminal commands
- Example with a CSV file
- Serialization and Deserialization
- Text vs Binary
- Image processing with a TIFF image

Files

An overview

A computer file is a resource for recording data on a computer storage device, primarily identified by its filename.

- ✓ Organized into one-dimensional arrays of *bytes*.
- ✓ Acts solely as a *container* for data (and metadata).

Using computer programs, we can —

- Open
- Read
- Write
- Change
- Save
- Close
- and perform many other operations on files.

data about
the data



The format of the file is indicated
by its filename extension.

File I/O

What is it and why we need it?

The functions under the umbrella of file I/O lets us *read* from/*write* to files.

Our programs so far have been a bit ephemeral!

They may —

- Perform some function
- Prompt the user for some input
- Print some output to the screen

But once the program **terminates**, there's no evidence that the output existed!

We need a means for storing *persistent* information.

 doesn't disappear when the program stops running!

File I/O

For the C language

C provides access on **high level functions** as well as **low level OSlevel calls** to handle file on our storage devices.

The abstraction of files that C provides is implemented in a data structure known as **FILE**.

- ✓ Almost universally when working with files, we'll be using pointers to them, **FILE***.

The file manipulation functions all live in **stdio.h**.

- All of them accept **FILE*** as one of their parameters, except for the function **fopen()**, which is used to get a file pointer in the first place.

Some of the most common file I/O functions that we'll be working with are:

fopen()	fclose()	fgetc()	fputc()
fread()	fwrite()	fscanf()	fprintf()

File I/O

The `fopen()` function

Opens a file and returns a **file pointer** to it.

Tip: Always check the return value to make sure you don't get back **NULL**.

```
FILE* ptr = fopen(<filename>, <operation>);
```

For example,

```
FILE* ptr1 = fopen("file1.txt", "r");  
FILE* ptr2 = fopen("file2.txt", "w");  
FILE* ptr3 = fopen("file3.txt", "a");
```

Read mode

Write mode

Append mode

File I/O

The `fopen()` function

There are a lot of other access modes.

- `r` — Opens an existing text file for reading purpose.
- `w` — Opens a text file for writing, if it does not exist then a new file is created. Here the program will start writing content from the beginning of the file.
- `a` — Opens a text file for writing in appending mode, if it does not exist then a new file is created. Here the program will start appending content with the existing file content.
- `r+` — Opens a text file for reading and writing both.
- `w+` — Opens a text file for reading and writing both. It first truncates the file to zero length if it exists otherwise create the file if it does not exist.
- `a+` — Opens a text file for reading and writing both. It creates the file if it does not exist. The reading will start from the beginning but writing can only be appended.

For binary files, we use `"rb"`, `"wb"`, `"ab"`, `"rb+"`, `"r+b"`, `"wb+"`, `"w+b"`, `"ab+"`, and `"a+b"` respectively.

File I/O

The `fclose()` function

Closes the file pointed to by the given file pointer.

```
fclose (<file pointer>) ;
```

It returns **zero** on success, or **EOF** if there is an error in closing the file.

It *flushes* any data still pending in the buffer to the file, *closes* the file, and *releases* any memory used for the file.

For example,

```
fclose (ptr1) ;
```

```
fclose (ptr2) ;
```

```
fclose (ptr3) ;
```


File I/O

The `fgetc()` function

Reads and returns the next character from the file pointed to.

Note: The operation of the file pointer passed in as a parameter must be `"r"` for read, or you will suffer an error.

```
char ch = fgetc(<file pointer>) ;
```

It returns the **character's ASCII value** on success, or **EOF** if there is an error.

For example,

```
char ch = fgetc(ptr1) ;
```

We have all the tools for reading the contents of a plaintext file.

Let's emulate the `cat` command from Linux!

File I/O

The cat command

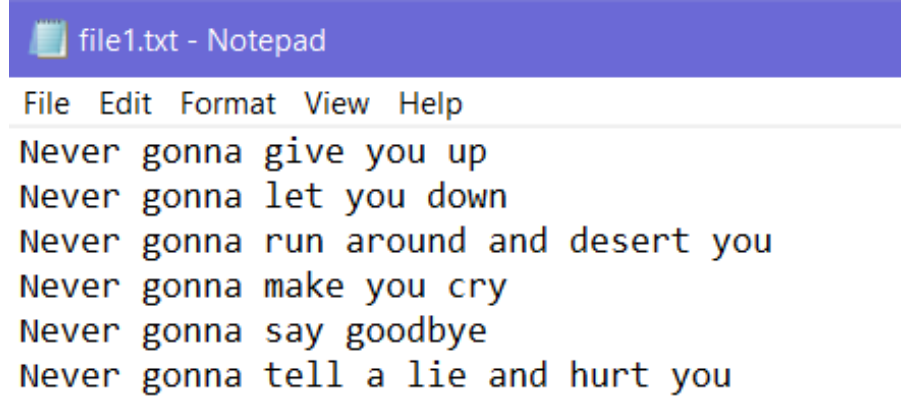
It is used to display the contents of a text file in the Linux terminal.

```
$ cat <filename>
```

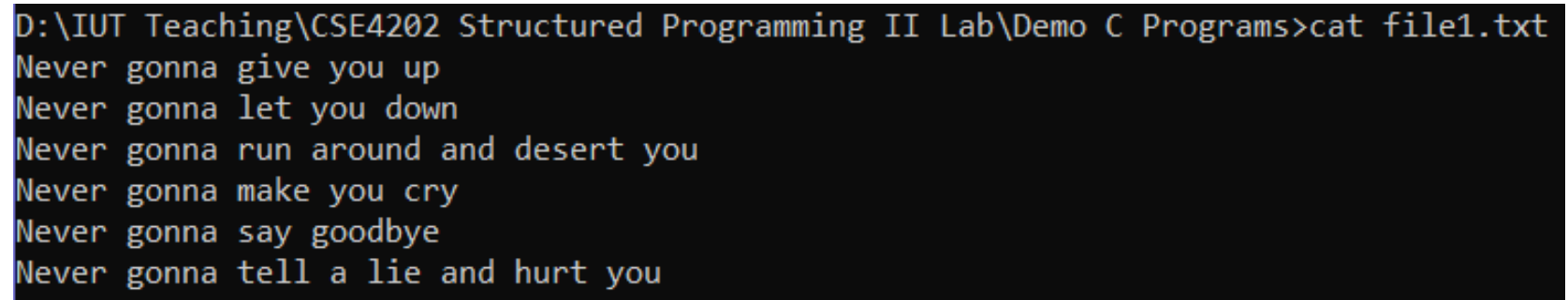
```
cat.c
#include <stdio.h>

int main(int argc, char *argv[])
{
    FILE *source = fopen(argv[1], "r");

    char ch;
    while(1)
    {
        ch = fgetc(source);
        if(ch == EOF)
        {
            break;
        }
        printf("%c", ch);
    }
    fclose(source);
    return 0;
}
```

A screenshot of a Notepad window titled "file1.txt - Notepad". The window has a menu bar with "File", "Edit", "Format", "View", and "Help". The text content of the file is displayed in a monospaced font: "Never gonna give you up", "Never gonna let you down", "Never gonna run around and desert you", "Never gonna make you cry", "Never gonna say goodbye", and "Never gonna tell a lie and hurt you".

```
file1.txt - Notepad
File Edit Format View Help
Never gonna give you up
Never gonna let you down
Never gonna run around and desert you
Never gonna make you cry
Never gonna say goodbye
Never gonna tell a lie and hurt you
```

A screenshot of a Linux terminal window. The prompt shows the current directory as "D:\IUT Teaching\CSE4202 Structured Programming II Lab\Demo C Programs". The command "cat file1.txt" has been executed, and the output is displayed in the terminal: "Never gonna give you up", "Never gonna let you down", "Never gonna run around and desert you", "Never gonna make you cry", "Never gonna say goodbye", and "Never gonna tell a lie and hurt you".

```
D:\IUT Teaching\CSE4202 Structured Programming II Lab\Demo C Programs>cat file1.txt
Never gonna give you up
Never gonna let you down
Never gonna run around and desert you
Never gonna make you cry
Never gonna say goodbye
Never gonna tell a lie and hurt you
```

File I/O

The `fputc()` function

Writes or appends the specified character to the pointed-to file.

Note: The operation of the file pointer passed in as a parameter must be `"w"` or `"a"`, otherwise you will suffer an error.

```
fputc(<character>, <file pointer>);
```

It returns the **character's ASCII value** on success, or **EOF** if there is an error.

For example,

```
fputc('a', ptr1);
```

We have all the tools for copying the contents of a plaintext file to another.

Let's emulate the `cp` command from Linux!

File I/O

The `cp` command (for *plaintext* files)

It is used for copying files from one location to another in the Linux terminal.

\$ cp <source filename> <destination filename>

```
cp1.c
#include <stdio.h>

int main(int argc, char *argv[])
{
    FILE *source = fopen(argv[1], "r");
    FILE *destination = fopen(argv[2], "w");
    // FILE *destination = fopen(argv[2], "a");

    char ch;
    while(1)
    {
        ch = fgetc(source);
        if(ch == EOF)
        {
            break;
        }
        // printf("%c", ch);
        fputc(ch, destination);
    }
    fclose(source);
    fclose(destination);
    return 0;
}
```

argv[0] argv[1] argv[2]

>cp1 file1.txt file2.txt

file1.txt - Notepad

File Edit Format View Help

Never gonna give you up
Never gonna let you down
Never gonna run around and desert you
Never gonna make you cry
Never gonna say goodbye
Never gonna tell a lie and hurt you

file2.txt - Notepad

File Edit Format View Help

Never gonna give you up
Never gonna let you down
Never gonna run around and desert you
Never gonna make you cry
Never gonna say goodbye
Never gonna tell a lie and hurt you

File I/O

The `fread()` function

Reads **<qty>** units of **<size>** sized elements from the file pointed to and stores them in memory in a buffer (usually an array) pointed to by **<buffer>**.

Note: The operation of the file pointer passed in as a parameter must be **"r"** for read, or you will suffer an error.

```
fread(<buffer>, <size>, <qty>, <file pointer>);
```

It returns the **number of elements** read successfully.

For example,

```
int arr[10];  
  
fread(arr, sizeof(int), 10, ptr);  
  
char c;  
  
fread(&c, sizeof(char), 1, ptr);
```

File I/O

The `fwrite()` function

Writes `<qty>` units of `<size>` sized elements to the file pointed to by reading them from a buffer (usually an array) pointed to by `<buffer>`.

Note: The operation of the file pointer passed in as a parameter must be `"w"` or `"a"`, or you will suffer an error.

```
fwrite(<buffer>, <size>, <qty>, <file pointer>);
```

It returns the **number of elements** written successfully.

For example,

```
int arr[10];  
  
fwrite(arr, sizeof(int), 10, ptr);  
  
char c;  
  
fwrite(&c, sizeof(char), 1, ptr);
```

File I/O

The `cp` command (for *binary* files)

```
$ cp <source filename> <destination filename>
```

`cp2.c`

```
#include <stdio.h>
#include <stdint.h>

typedef uint8_t BYTE;

int main(int argc, char *argv[])
{
    FILE *source = fopen(argv[1], "rb");
    FILE *destination = fopen(argv[2], "wb");
    // FILE *destination = fopen(argv[2], "ab");

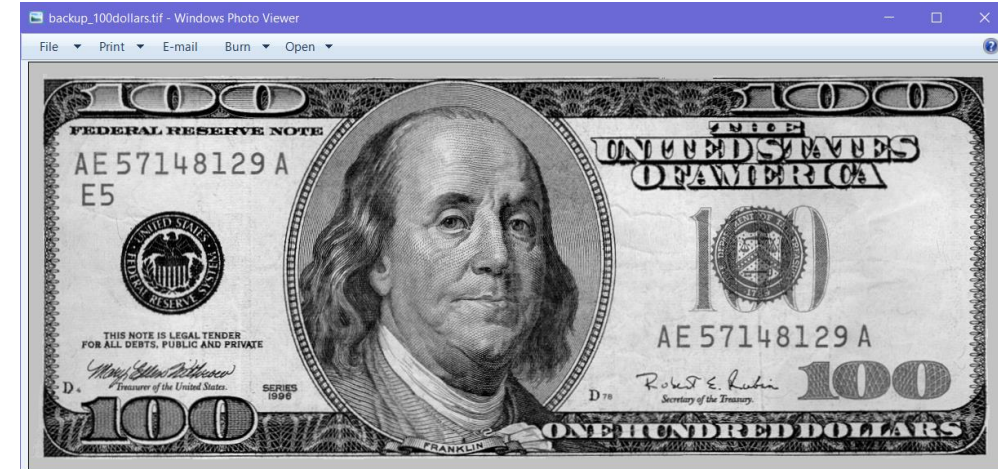
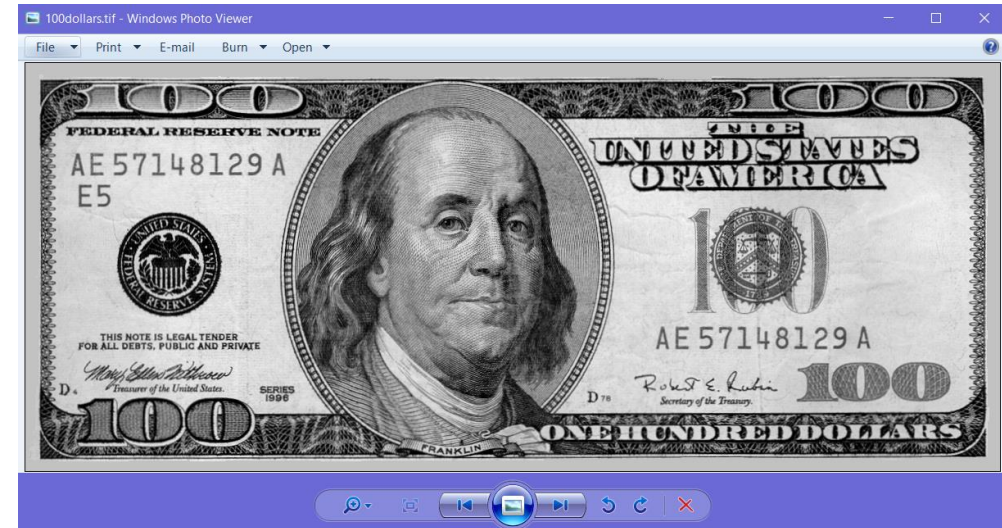
    BYTE b;
    int bytes = 0;
    while(fread(&b, sizeof(b), 1, source) != 0)
    {
        fwrite(&b, sizeof(b), 1, destination);
        // bytes++;
    }

    fclose(source);
    fclose(destination);
    // printf("%d", bytes);

    return 0;
}
```

`argv[0]` `argv[1]` `argv[2]`

```
>cp2 100dollars.tif backup_100dollars.tif
```



File I/O

A plethora of other functions

Lots of other useful functions abound in **stdio.h** for you to work with.

Here are some of the ones you may find useful!

- **fgets ()** — Reads a full string from a file.
- **fputs ()** — Writes a full string to a file.
- **fscanf ()** — Reads a formatted string from a file.
- **fprintf ()** — Writes a formatted string to a file.
- **fseek ()** — Allows you rewind or fast-forward within a file. (like an implicit byte position cursor!)
- **ftell ()** — Tells you at what (byte) position you are at within a file.
- **feof ()** — Tells you whether you've read to the end of a file.
- **ferror ()** — Indicates whether an error has occurred in working with a file.

File I/O

Working with a CSV file

`students.c`

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *file = fopen("students.csv", "a");
```

```
    char *name;
```

```
    int studentID;
```

```
    printf("Name: ");
```

```
    scanf("%s", name);
```

```
    printf("Student ID: ");
```

```
    scanf("%d", &studentID);
```

```
    fprintf(file, "%d,%s\n", studentID, name);
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```

"D:\IUT Teaching\CSE4202 Structured Programming II Lab\Demo C Programs\students.exe"

Name: Rifat

Student ID: 180041205

Process returned 0 (0x0) execution time : 10.056 s
Press any key to continue.

"D:\IUT Teaching\CSE4202 Structured Programming II Lab\Demo C Programs\students.exe"

Name: Ishrak

Student ID: 160041051

Process returned 0 (0x0) execution time : 9.817 s
Press any key to continue.

"D:\IUT Teaching\CSE4202 Structured Programming II Lab\Demo C Programs\students.exe"

Name: Maksud

Student ID: 180041222

Process returned 0 (0x0) execution time : 10.578 s
Press any key to continue.

students.csv - Notepad

File Edit Format View Help

180041205,Rifat

160041051,Ishrak

180041222,Maksud

|

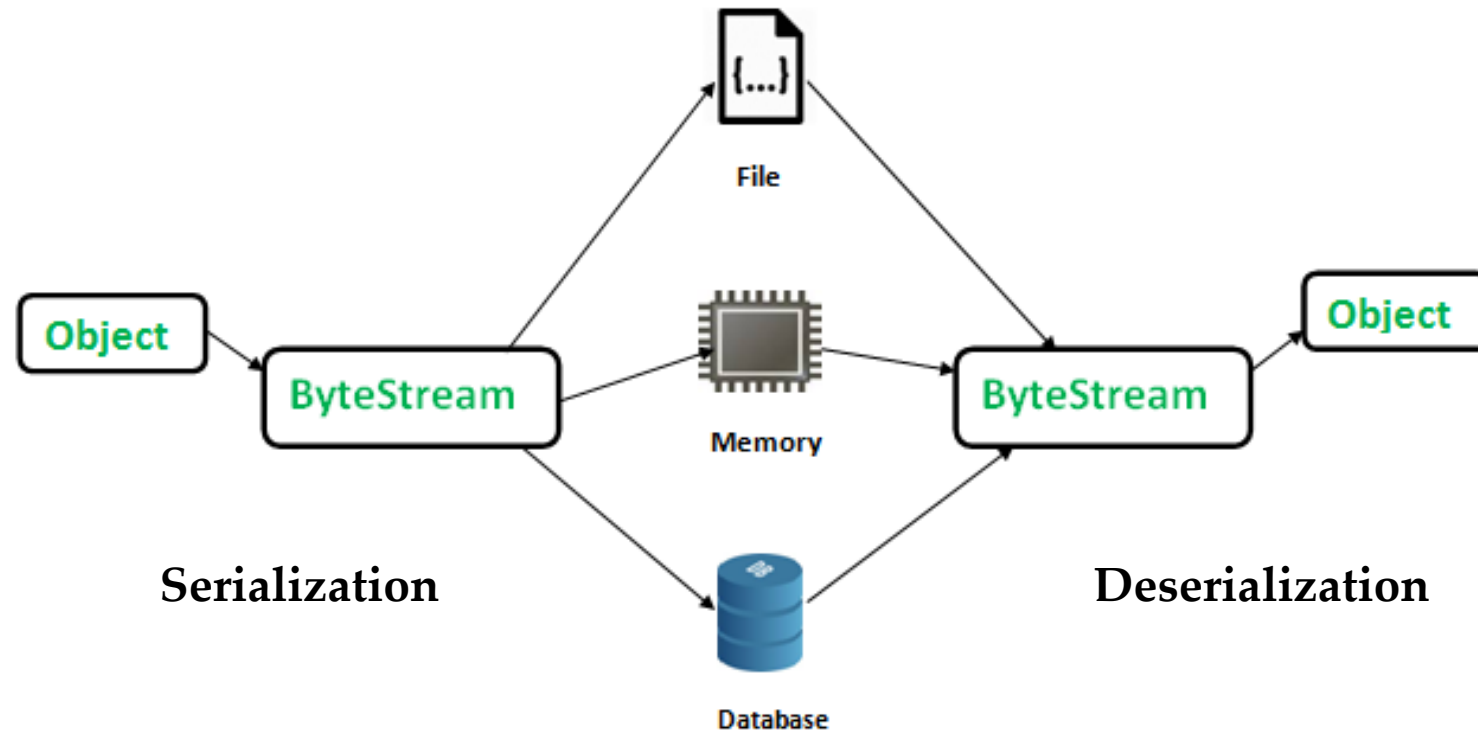
	A	B	C
1	180041205	Rifat	
2	160041051	Ishrak	
3	180041222	Maksud	
4			
5			

Serialization & Deserialization

What are they?

Serialization is the process of converting a **struct** or an object into a stream of bytes to store it in a file or memory.

Deserialization is the exact opposite—we read the contents from the file or memory and store it in an instance of the **struct** or object.



Serialization

For text files

```
text_serialization.c
```

```
#include <stdio.h>
#include <string.h>
```

```
typedef struct Student
{
    char name[10];
    int studentID;
} Student;
```

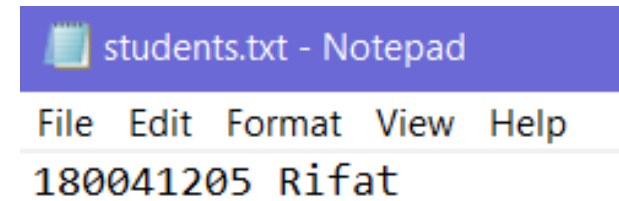
```
int main()
{
    Student s1;

    strcpy(s1.name, "Rifat");
    s1.studentID = 180041205;

    FILE *file = fopen("students.txt", "w");
    fprintf(file, "%d %s", s1.studentID, s1.name);

    fclose(file);

    return 0;
}
```



Deserialization

For text files

text_deserialization.c

```
#include <stdio.h>
#include <string.h>
```

```
typedef struct Student
{
    char name[10];
    int studentID;
} Student;
```

```
int main()
{
    Student s2;

    FILE *file = fopen("students.txt", "r");
    fscanf(file, "%d %s", &s2.studentID, s2.name);

    fclose(file);

    printf("Name: %s\n", s2.name);
    printf("Student ID: %d\n", s2.studentID);

    return 0;
}
```

students.txt - Notepad

File Edit Format View Help

180041205 Rifat

"D:\IUT Teaching\CSE4202 Structured Programming II Lab\Demo C Programs\text_deserialization.exe"

Name: Rifat
Student ID: 180041205

Process returned 0 (0x0) execution time : 1.914 s
Press any key to continue.

Serialization

For binary files

```
bin_serialization.c
#include <stdio.h>
#include <string.h>

typedef struct Student
{
    char name[10];
    int studentID;
} Student;

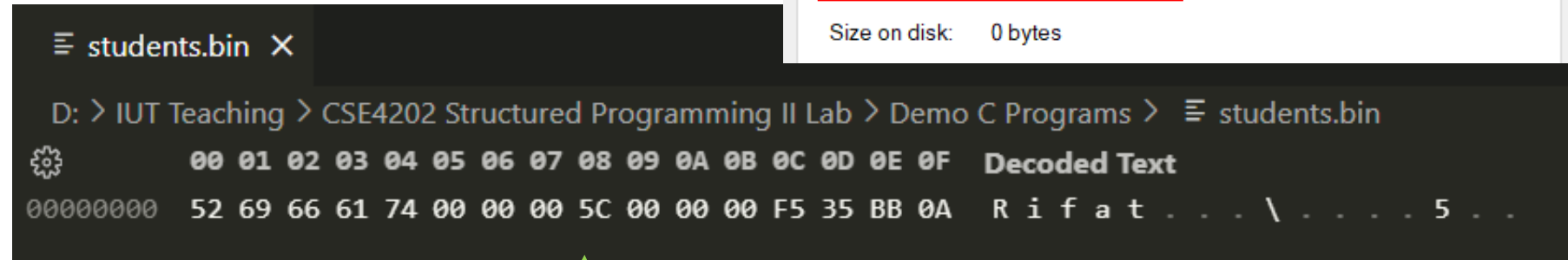
int main()
{
    Student s1;

    strcpy(s1.name, "Rifat");
    s1.studentID = 180041205;

    FILE *file = fopen("students.bin", "wb");
    fwrite(&s1, sizeof(Student), 1, file);

    fclose(file);

    return 0;
}
```



8 byte word for 64-bit machine



Total = 8 + 8 = 16 bytes

Deserialization

For binary files

```
bin_deserialization.c
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
typedef struct Student  
{
```

```
    char name[10];
```

```
    int studentID;
```

```
} Student;
```

```
int main()
```

```
{
```

```
    Student s2;
```

```
    FILE *file = fopen("students.bin", "rb");
```

```
//    FILE *file = fopen("students_altered.bin", "rb");
```

```
    fread(&s2, sizeof(Student), 1, file);
```

```
    fclose(file);
```

```
    printf("Name: %s\n", s2.name);
```

```
    printf("Student ID: %d\n", s2.studentID);
```

```
    return 0;
```

```
}
```

```
students.bin X  
D: > IUT Teaching > CSE4202 Structured Programming II Lab > Demo C Programs > students.bin  
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded Text  
00000000 52 69 66 61 74 00 00 00 5C 00 00 00 F5 35 BB 0A R i f a t . . . \ . . . . 5 . .
```

```
"D:\IUT Teaching\CSE4202 Structured Programming II Lab\Demo C Programs\bin_deserialization.exe"  
Name: Rifat  
Student ID: 180041205  
Process returned 0 (0x0)   execution time : 1.378 s  
Press any key to continue.
```

Deserialization

For binary files (*altering* information)

```
bin_deserialization.c
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
typedef struct Student
```

```
{
```

```
    char name[10];
```

```
    int studentID;
```

```
} Student;
```

```
int main()
```

```
{
```

```
    Student s2;
```

```
//    FILE *file = fopen("students.bin", "rb");
```

```
FILE *file = fopen("students_altered.bin", "rb");
```

```
fread(&s2, sizeof(Student), 1, file);
```

```
fclose(file);
```

```
printf("Name: %s\n", s2.name);
```

```
printf("Student ID: %d\n", s2.studentID);
```

```
return 0;
```

```
}
```

```
students_altered.bin X
D: > IUT Teaching > CSE4202 Structured Programming II Lab > Demo C Programs > students_altered.bin
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded Text
00000000 53 69 66 61 74 00 00 00 5C 00 00 00 FF 35 BB 0A S i f a t . . . \ . . . . 5 . .
```

```
"D:\IUT Teaching\CSE4202 Structured Programming II Lab\Demo C Programs\bin_deserialization.exe"
Name: Sifat
Student ID: 180041215
Process returned 0 (0x0)    execution time : 1.308 s
Press any key to continue.
```

Text vs Binary

What are the differences?

For **plaintext** files,

- We use formatted string for serialization/deserialization.
- Comparatively **slower** since the number of bytes read/written depends on the length of the content (*e.g.* 10 bytes for a 10-digit number).
- Generally **compatible** across different architectures.

For **binary** files,

- We directly read/write the bytes for serialization/deserialization.
- Comparatively **faster** since the number of bytes read/written depends on the inherent datatypes (*e.g.* 4 bytes for a 10-digit number).
- May not be compatible across different architectures (*e.g.* **inconsistent** pointer size, **inconsistent** `int` size, etc.).

Serialization

For struct arrays

struct_serialization.c

```
typedef struct Student
{
    char name[10];
    int studentID;
    int semester;
    double cgpa;
    char phone[12];
    char address[10];
} Student;
```

```
Student s[3];

strcpy(s[0].name, "Ishmam");
s[0].studentID = 180041105;
s[0].semester = 8;
s[0].cgpa = 3.97;
strcpy(s[0].phone, "01764370668");
strcpy(s[0].address, "Uttara");

strcpy(s[1].name, "Ishrak");
s[1].studentID = 160041051;
s[1].semester = 8;
s[1].cgpa = 3.99;
strcpy(s[1].phone, "01677169586");
strcpy(s[1].address, "Mirpur");

strcpy(s[2].name, "Atiq");
s[2].studentID = 180041123;
s[2].semester = 8;
s[2].cgpa = 3.98;
strcpy(s[2].phone, "01780420405");
strcpy(s[2].address, "Gazipur");
```

Serialization

For struct arrays

struct_serialization.c

```
typedef struct Student
{
    char name[10];
    int studentID;
    int semester;
    double cgpa;
    char phone[12];
    char address[10];
} Student;
```

```
const char* STUDENT_WRITE_FORMAT = "{\"name\": \"%s\", \
    \"academicInfo\": \
        {\"studentID\": %d, \
        \"semester\": %d, \
        \"cgpa\": %lf}, \
    \"personalInfo\": \
        {\"phone\": \"%s\", \
        \"address\": \"%s\"}}\
    },\";

FILE *file = fopen(\"students.json\", \"w\");

fprintf(file, \"[\n\n\");
for(int i = 0; i < 3; i++)
{
    fprintf(file, STUDENT_WRITE_FORMAT, s[i].name, \
        s[i].studentID, s[i].semester, s[i].cgpa, \
        s[i].phone, s[i].address);
}
fprintf(file, \"\n]\");

fclose(file);

return 0;
```

Serialization

For struct arrays

Not properly formatted yet.

```
{ } students.json 1 X
D: > IUT Teaching > CSE4202 Structured Programming II Lab > Demo C Programs > { } students.json > ...
1  [
2
3  {"name": "Ishmam", "academicInfo": {"studentID": 180041105, "semester": 8, "cgpa": 3.970000}, "personalInfo": {"phone": "01764370668", "address": "Uttara"}},
4  {"name": "Ishrak", "academicInfo": {"studentID": 160041051, "semester": 8, "cgpa": 3.990000}, "personalInfo": {"phone": "01677169586", "address": "Mirpur"}},
5  {"name": "Atiq", "academicInfo": {"studentID": 180041123, "semester": 8, "cgpa": 3.980000}, "personalInfo": {"phone": "01780420405", "address": "Gazipur"}},
6
7  ]
8
```

Serialization

For struct arrays

With proper JSON formatting,
it becomes quite readable.

```
{
  "name": "Ishmam",
  "academicInfo": {
    "studentID": 180041105,
    "semester": 8,
    "cgpa": 3.970000
  },
  "personalInfo": {
    "phone": "01764370668",
    "address": "Uttara"
  }
},
{
  "name": "Ishrak",
  "academicInfo": {
    "studentID": 160041051,
    "semester": 8,
    "cgpa": 3.990000
  },
  "personalInfo": {
    "phone": "01677169586",
    "address": "Mirpur"
  }
},
{
  "name": "Atiq",
  "academicInfo": {
    "studentID": 180041123,
    "semester": 8,
    "cgpa": 3.980000
  },
  "personalInfo": {
    "phone": "01780420405",
    "address": "Gazipur"
  }
},
}
```

Student s[0]

Student s[1]

Student s[2]

Deserialization

For struct arrays

struct_deserialization.c

```
typedef struct Student
{
    char name[10];
    int studentID;
    int semester;
    double cgpa;
    char phone[12];
    char address[10];
} Student;
```

```
const char* STUDENT_READ_FORMAT = "{\"name\": \"%[^\"]\", \"academicInfo\": {\"studentID\": %d, \"semester\": %d, \"cgpa\": %lf}, \"personalInfo\": {\"phone\": \"%[^\"]\", \"address\": \"%[^\"]\"}}\", \n";

Student s[3];
FILE *file = fopen("students.json", "r");
fscanf(file, "[\n\n");
for(int i = 0; i < 3; i++)
{
    fscanf(file, STUDENT_READ_FORMAT, s[i].name, \
        &s[i].studentID, &s[i].semester, &s[i].cgpa, \
        s[i].phone, s[i].address);
}
fscanf(file, "\n]");

fclose(file);
```

Deserialization

For struct arrays

`struct_deserialization.c`

```
for(int i = 0; i < 3; i++)
{
    printf("Name: %s\n", s[i].name);
    printf("Student ID: %d\n", s[i].studentID);
    printf("Semester: %d\n", s[i].semester);
    printf("CGPA: %lf\n", s[i].cgpa);
    printf("Phone: %s\n", s[i].phone);
    printf("Address: %s\n", s[i].address);
    printf("\n");
}
return 0;
```

```
"D:\IUT Teaching\CSE4202 Structured Programming II Lab\Demo C Programs\struct_deserialization.exe"
Name: Ishmam
Student ID: 180041105
Semester: 8
CGPA: 3.970000
Phone: 01764370668
Address: Uttara

Name: Ishrak
Student ID: 160041051
Semester: 8
CGPA: 3.990000
Phone: 01677169586
Address: Mirpur

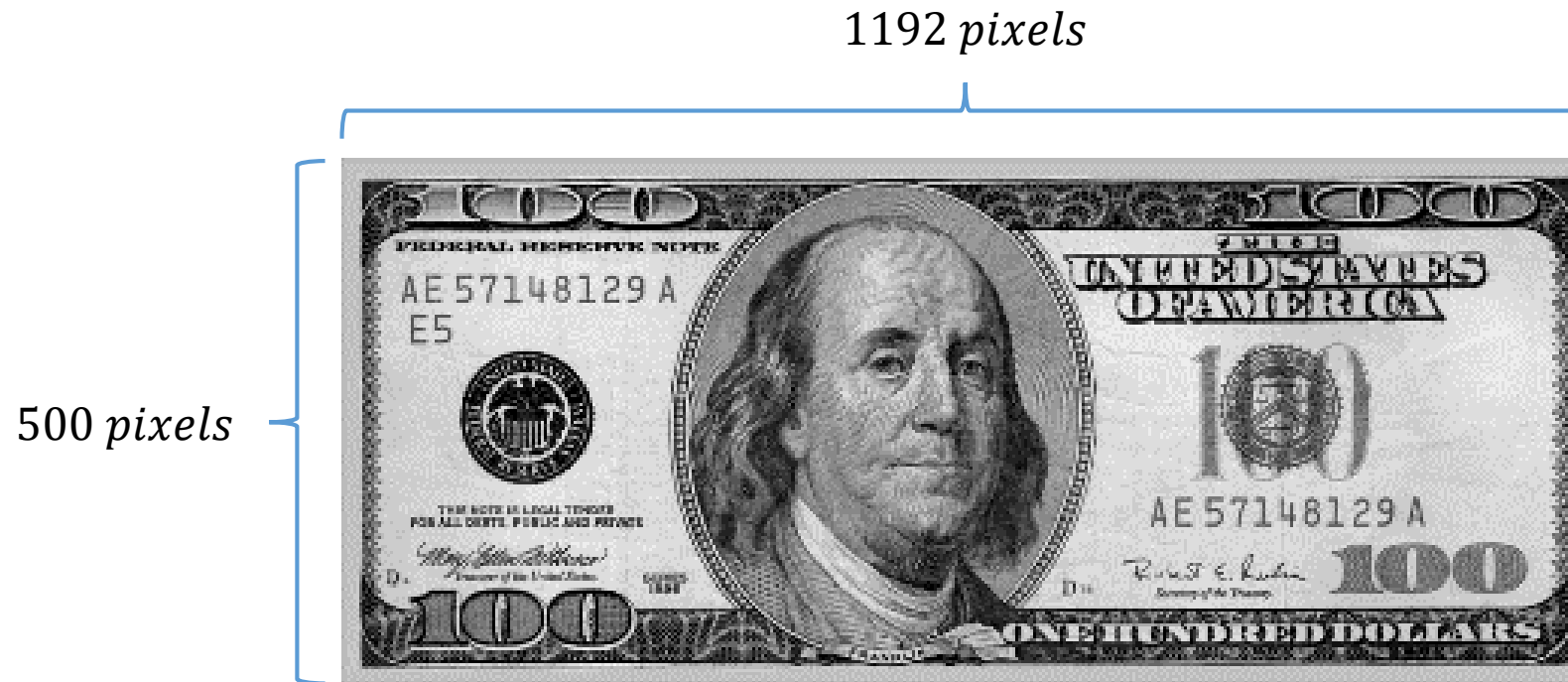
Name: Atiq
Student ID: 180041123
Semester: 8
CGPA: 3.980000
Phone: 01780420405
Address: Gazipur

Process returned 0 (0x0)    execution time : 1.836 s
Press any key to continue.
```

Image Processing with TIFF Images

An overview about TIFF

A TIFF, which stands for Tag Image File Format, is a computer file used to store raster graphics and image information.



8 byte Metadata

$M \times N$ bytes pixel content

Metadata

Image Processing with TIFF Images

Your assignment

1192 *pixels*

Flip and blur!

500 *pixels*

